Dawn Technology
www.dawn.tech

The Frontend(react) test

Candidate sample application

Auteur: Dawn Technology

Versie: 1.0

Contact: quinton.verbon@dawn.tech gerard.kock@dawn.tech

Candidate sample application

Introduction

At Dawn Technology we work on challenging projects for a range of large enterprise clients. To make sure that the developers working on these projects have the right skills and knowledge, our recruitment process is very thorough.

In preparation for the interview with one of our senior developers, we ask you to build a small sample application. This doesn't have to be a rock-solid, highly scalable or production-ready application, but it should give us a clear idea of your architectural choices, state management, and coding standards.

The goal is to show what you are capable of, particularly in how you handle business logic, reactivity, and asynchronous data within a modern frontend framework.

There are no strict limits on the amount of time you spend on this, but try to keep things simple while demonstrating your seniority. If you use specific frameworks (React, Vue, Angular), libraries, or state-management tools, please mention them in your documentation.

Once you've finished the application, please send your results (ideally a repository link) to your recruitment contact.

Requirements

You are required to create an **Order Management Dashboard**. This application allows users to track and manage shipping orders for a fictional logistics company.

Data Model: Each Order must consist of the following information:

- **Destination Country**
- **Shipping Date**
- **Price**

Features:

1. **Dashboard Page:** A summary view that displays:
    - Total number of orders.
    - Total price of all orders combined.
    - Number of unique countries served.
2. **Order Overview Page:** A view where all shipping orders can be easily viewed in a list or table. From this page:
    - A user can **Create** a new order.
    - A user can **Update** an existing order.
    - A user can **Delete** an existing order.

Technical Note: You do not need to build a backend. You may use local state, `localStorage`, or a mocked API. We are interested in how you manage the "Source of Truth" when data is updated or deleted across different views.

We are looking for a solution that demonstrates **Senior-level engineering**. While the requirements above are simple, we will evaluate your submission based on how you handle the "hidden" complexities of a professional frontend application.

Specifically, we want to see how you handle:

- **Data Integrity:** Ensuring that changes made in the Inventory (Create/Update/Delete) are accurately and immediately reflected in the Analytics Dashboard.
- **Architecture:** How you organize your code to be maintainable, readable, and scalable.
- **Resilience:** How the application behaves during user errors or empty data states.
- **Professional Standards:** We expect the code to be production-ready in terms of structure and reliability.

Flowchart

The application should manage the data lifecycle as follows:

1. Start: Load initial application state.
2. Dashboard: Display calculated metrics (Totals, Unique Countries).
3. Inventory: Provide the user with CRUD capabilities.
4. Mutation: When data is added, changed, or removed:
    - Update the central state.
    - Sync the changes across all pages/views.
5. Stop.